Quran Recitation AI

Related Works Survey & Technical Glossary

A structured collection of paper summaries,
technical breakdowns, and key terminology
for the development of an AI model
for Quranic recitation with Tajweed.

Research Notes

March 4, 2026

Contents

Part I

Paper Summaries

Chapter 1

Birgün (2026) — Integrating AI into Qur'an Learning

Paper Metadata

Title	Integrating AI into Qur'an learning: Technical advances and pedagogical gaps
Author	Mehmet Birgün
Journal	Social Sciences & Humanities Open, Vol. 13 (2026), Article 102499
Type	Qualitative review + comparative application study (no original model)
DOI	10.1016/j.ssaho.2026.102499

1.1 Objective

The paper maps the current state of AI-based tools for Qur'an recitation learning. It does **not** propose a new model; instead it surveys the literature and evaluates five existing applications, then proposes a conceptual framework (*Fem-i Muhsin AI*) as a roadmap for future development.

1.2 Methodology

1.2.1 Study Design

Qualitative document review combining two data sources:

1. **Literature review** — peer-reviewed articles (2019–2025) from Google Scholar, ERIC, SpringerLink, ScienceDirect, ResearchGate. Keywords: *Qur'an education, artificial intelligence, tajweed detection, AI recitation, adaptive learning.*
2. **Application review** — comparative analysis of 5 apps selected by: (a) availability on App Store/Google Play, (b) ≥500k downloads or ≥4.0 rating, (c) focus on

pronunciation and tajweed.

1.2.2 Analysis Method

Descriptive thematic coding around three themes: **technical accuracy**, **pedagogical contribution**, and **research gaps**.

1.3 Key Technical Findings from Reviewed Studies

Study	Approach	Key Result / Limitation
Harere & Jallad (2023b)	Deep learning on QDAT dataset	95–96% accuracy for mispronunciation detection; instructional feedback generation absent
Shaiakhmetov et al. (2025)	DNN classifying Al Mad, Ghunnah, Ikhfaa	95–99% accuracy; purely technical, no pedagogical output
Alrashoudi et al. (2025)	Transformer-based Arabic MDD; errors typed as insertions / deletions / substitutions	Exceeded human raters on phoneme-level detection; no correction pathways
Hadwan et al. (2023)	CNN–Transformer hybrid ASR with RNN language model	CER 1.98%, WER 6.16% on diacritized verses; no tajweed guidance
Kheir et al. (2025)	SSL-based baseline models	Released *QuranMB.v1* — first unified benchmark for Qur'anic mispronunciation detection
Refaee (2024)	Google Teachable Machine; visual/colour-coded classification	Automatically extracts Idgham, Qalqalah via visual cues; preliminary, unpublished
Hassan et al. (2025)	Multimodal fusion (VSCF)	97.68% reciter classification accuracy; limited to acoustic classification, no tajweed feedback

1.4 Application Comparative Review

App	Main Functions	Strengths	Limitations
Tarteel	Speech recognition, verse verification, memorisation tracking	Large user base; instant feedback; progress monitoring	No rule-based explanations (e.g. madd rules); monochrome error display; limited letter-level feedback
Qur'an Tutor	Pronunciation correction, practice exercises	User-friendly interface; accessible design	Feedback superficial; no step-by-step correction guide; tajweed reasoning minimal
TajweedMate	Tajweed rule detection, visual colour markers	Strongest tajweed-focused UI; colour-coded symbols	Inconsistent speech recognition; unreliable error detection
Qur'an University	Student–teacher interaction, assignments, testing	Strong pedagogical integration; institutional framework	AI contribution minimal; essentially a conventional LMS
Qur'an Mobasher	Live teacher-led audio/video	Combines human tutoring with digital tools; one-on-one learning	Not AI-based; fully dependent on human instructors

1.5 Proposed Conceptual Framework — *Fem-i Muhsin AI*

The paper proposes (but does not implement) a learning cycle with three pillars:

1. **Technical Accuracy** — detect mispronunciation and tajweed violations
2. **Pedagogical Depth** — provide explanatory feedback, model correct recitation, guide practice
3. **Spiritual & Social Context** — positive reinforcement, community learning integration

The cycle flow is:

Learner recites → AI Analysis & Diagnosis → Explanatory Feedback → AI Modelling → Guide

Theoretical frameworks used: **TPACK** (Technological Pedagogical Content Knowledge) and **Adaptive Learning Theory**.

1.6 Identified Gaps (Relevant to Your Project)

Critical Gaps Highlighted

- No existing system handles **maqam-based recitation synthesis** (Rast, Segah, Hijaz, Saba)
- All reviewed systems detect errors but **cannot explain how to correct** them
- **Semantic understanding** (word meaning, context) is absent in all tools
- **Adaptive personalised learning paths** do not exist yet in any Qur'an app
- No large publicly available dataset covering multiple *qira'at*, tones, dialects, and tajweed labels

1.7 Relevance to Your Research

This paper is primarily useful as a **survey reference** and **gap justification** for your introduction/related works section. It confirms that:

- The field is technically advanced in *detection* but weak in *correction* and *pedagogy*
- No existing tool reaches a "Fem-i Muhsin" level of instruction
- Your model should aim beyond binary correct/incorrect labels toward **rule-specific, actionable feedback**

Chapter 2

Hassan et al. (2025) — Advanced Voice Classification via Multimodal Fusion

Paper Metadata

Title	An innovative approach to advanced voice classification of sacred Quranic recitations through multimodal fusion
Authors	Esraa Hassan, Abeer Saber, Omar Alqahtani, Nora El-Rashidy, Samar Elbedwehy
Journal	Egyptian Informatics Journal, Vol. 30 (2025), Article 100640
Type	Original model — reciter identification (speaker classification)
DOI	10.1016/j.eij.2025.100640

Important Scope Note

This model classifies **who** is reciting (reciter identity), **not** whether the recitation is correct. It is **not** a tajweed error detector. Its architecture is however directly reusable for tajweed classification if retrained on labelled tajweed data.

2.1 Objective

Build a robust automatic voice classification system for Quranic recitations that can identify which of 12 reciters is speaking, while handling challenges of dataset size, reciter variation, and acoustic diversity.

2.2 Dataset

Quranic Recitation Audio Classification Dataset

Total files	7,144 WAV audio files
Reciters	12 (e.g. Maher Almuaiqly, AbdulRahman Alsudais, Saad Alghamdi . . .)
Sampling rate	22,050 Hz
Split	80% training / 20% testing
Format	One folder per reciter (class label = folder name)

2.3 Proposed Architecture — VSCF (Voice Shortcut Connection Fusion)

The model has **two parallel branches** that are fused before the classification head.

2.3.1 Branch 1 — Acoustic Model (MFCC + CNN)

What it does

Converts raw audio into compact numerical descriptors that represent the spectral and temporal characteristics of the voice at a fine-grained (phoneme) level.

Step-by-step pipeline

1. **Load audio signal** $s(t)$
   The raw WAV file is loaded at 22,050 Hz — meaning 22,050 amplitude samples per second of audio.
2. **FFT (Fast Fourier Transform)**
   Converts the time-domain signal into the frequency domain, revealing which frequencies are active at each moment. This is the foundation for all subsequent spectral analysis.
3. **Mel Filter Banks**
   Applies a bank of triangular filters spaced on the *Mel scale* — a perceptual scale that mimics how the human ear perceives frequency. The ear is more sensitive to low frequencies than high ones, so Mel spacing is denser at low frequencies. This step makes the features more relevant to human speech perception.
4. **Log compression**
   Takes the logarithm of the filter bank energies, producing the *Log Mel Spectrogram*. This compresses the dynamic range and makes the representation more robust to volume variations.

5. **DCT (Discrete Cosine Transform)**
   Decorrelates the filter bank outputs and compresses them into a small set of coefficients. The first 13 coefficients (MFCCs 1–13) capture the most important spectral envelope information while discarding redundant detail.

6. **Zero-padding**
   Applied to standardise input dimensions across audio clips of varying lengths, ensuring consistent tensor shapes for the neural network.

7. **Output: 13 MFCC coefficients per frame**
   These 13 numbers encode: vocal tract shape, phoneme identity, timbre, tone, and speaker-specific acoustic properties. Input tensor shape: `(None, 13, 1, 1)`.

CNN architecture on top of MFCCs

```
Input: (None, 13, 1, 1)
  -> Conv2D(32 filters, kernel 3x1, padding='same') + BatchNorm
     + ReLU
  -> Conv2D(32 filters, kernel 3x1, padding='same') + BatchNorm
     + ReLU
  -> MaxPooling2D(pool_size=(2,1))
  -> Dropout(0.3)
  -> Conv2D(64 filters, kernel 3x1, padding='same') + BatchNorm
     + ReLU
  -> MaxPooling2D(pool_size=(2,1))
  -> Dropout(0.5)
  -> Flatten  -> shape: (None, 1)
```

Listing 2.1: Acoustic CNN sub-network

Each layer's role:

- **Conv2D (32 filters)** — learns low-level patterns: basic phoneme transitions, short-range spectral contours
- **BatchNorm** — normalises activations layer-by-layer, preventing gradient issues and speeding up training
- **MaxPooling (2×1)** — reduces spatial dimensions by keeping only the dominant feature activations, providing translation invariance
- **Dropout (0.3 / 0.5)** — randomly deactivates neurons during training to prevent over-fitting
- **Conv2D (64 filters)** — learns higher-level combinations of low-level patterns (complex phoneme clusters)
- **Flatten** — converts the 2D feature maps into a 1D vector for fusion

In plain terms: This branch *listens* to the raw sound and extracts a compact finger-

print of the voice's fine-grained phonetic characteristics.

2.3.2 Branch 2 — Linguistic Model (ResNet50 + Global Average Pooling)

What it does

Extracts high-level, abstract representations that go beyond raw sound — capturing long-range stylistic, rhythmic, and structural patterns that define a reciter's "signature".

Why ResNet50?

ResNet50 is a 50-layer deep residual convolutional network originally designed for image classification. It is used here via **transfer learning**: the network's weights pre-trained on large data are reused, and only the final classification layer is removed. Its key innovation is the **residual (skip) connection**:

$$\text{output} = F(x) + x$$

where $F(x)$ is what the layers learn and x is the original input passed directly forward. This solves the *vanishing gradient* problem in very deep networks and allows learning of very fine residual differences.

Architecture inside ResNet50 (adapted)

```
Input: audio representation
  -> cfg[0] residual blocks: 64  filters, output 56x56
  -> cfg[1] residual blocks: 128 filters, output 28x28
  -> cfg[2] residual blocks: 256 filters, output 14x14
  -> cfg[3] residual blocks: 512 filters, output  7x7
  -> [Last classification layer removed]
  -> Global Average Pooling  -> shape: (None, 512)
```

Listing 2.2: ResNet50 linguistic sub-network

Each residual block internally does:

```
x_input
  -> Conv(1x1) -> BN -> ReLU
  -> Conv(3x3) -> BN -> ReLU
  -> Conv(1x1) -> BN
  -> Add(x_input)       # skip connection
  -> ReLU
```

Listing 2.3: Inside one residual block

Global Average Pooling (GAP)

Instead of flattening (which would produce a huge vector of $7 \times 7 \times 512 = 25{,}088$ values), GAP takes the spatial mean of each of the 512 feature maps, producing a compact `(None, 512)` vector. This dramatically reduces parameters and acts as a strong regulariser.

What features does ResNet50 capture?

- Long-range melodic and rhythmic patterns in the recitation
- Reciter-specific stylistic signatures
- Abstract structural patterns not visible in frame-level MFCCs

In plain terms: While the MFCC branch *hears* the sound, the ResNet50 branch *recognises the style* — more like identifying a "musical fingerprint" of the reciter.

2.3.3 Fusion Layer

What it does

Merges both branches into a single unified representation.

$$X_{\text{acoustic}} \in \mathbb{R}^1, \quad X_{\text{linguistic}} \in \mathbb{R}^{512}$$

$$F = \text{Concatenate}(X_{\text{acoustic}},\ X_{\text{linguistic}}) \in \mathbb{R}^{513}$$

Why concatenation and not addition/averaging?
Concatenation preserves all information from both branches independently. The subsequent dense layer then learns which dimensions of each branch are most discriminative. Addition would risk cancellation of complementary features.

What fusion achieves:

- MFCCs alone miss long-range stylistic patterns
- ResNet50 alone misses fine-grained phonetic detail
- Together: both micro (phoneme-level) and macro (style-level) representations are available

2.3.4 Classification Head

```
Fused vector: (None, 513)
  -> Dense(256) + BatchNorm + ReLU
  -> Dense(12)  + Softmax        # 12 reciter classes
```

Listing 2.4: Classification head

Softmax outputs a probability distribution: p_i = probability the audio belongs to reciter i, with $\sum_{i=1}^{12} p_i = 1$.

2.3.5 Full Data Flow Diagram

```
WAV Audio
  |
  +------------------------------+
  |                              |
  v                              v
MFCC Extraction            ResNet50 (pretrained, last layer
   removed)
(13 coefficients/frame)    (transfer learning)
  |                              |
  v                              v
CNN layers                 Residual blocks
(32 -> 64 filters)         (64 -> 128 -> 256 -> 512 filters)
  |                              |
  v                              v
Flatten (shape: 1)         Global Avg Pooling (shape: 512)
  |                              |
  +------------+-----------------+
               |
               v
         Concatenate (513)
               |
               v
          Dense(256) + BN + ReLU
               |
               v
         Dense(12) + Softmax
               |
               v
        Reciter Identity (class 0-11)
```

Listing 2.5: End-to-end VSCF pipeline

2.4 Results

VSCF Performance

Accuracy	97.683%
Sensitivity	97.52%
Specificity	97.85%
Precision	98.75%
F1 Score	98.13%

2.4.1 Comparison with Baselines (same dataset, 50 epochs)

Architecture	Optimizer	Accuracy
VSCF (proposed)	**Adamax**	**97.683%**
CNN	Adam	96.487%
CNN	Nadam	96.180%
WaveNet-inspired autoencoder	Adam	95.880%
CNN	RMSprop	93.790%
CNN	SGD	92.825%
CNN	Adagrad	76.457%
WaveNet-inspired autoencoder	SGD	11.136%
CNN	Adadelta	11.136%

2.5 Limitations

- **Task mismatch:** classifies reciter identity, not recitation correctness or tajweed compliance
- **Dataset size:** 7,144 files from 12 reciters — limited demographic diversity (gender imbalance noted)
- **No pedagogical output:** the model produces a class label only, with no feedback or explanation
- **Fixed audio duration:** 3-second clips may not capture full tajweed rule contexts

2.6 Relevance to Your Research

How to reuse this architecture

- **Replace the output layer:** swap Dense(12 reciters) with Dense(N tajweed rule classes) or a binary correct/incorrect head
- **Replace the dataset:** need audio clips labelled with tajweed rule violations rather than reciter identity
- **Keep the MFCC + fusion backbone:** the dual-branch fusion architecture is directly applicable — MFCCs capture phoneme-level errors, ResNet50 captures longer-range recitation patterns
- **Extend feedback:** add a decoder or explanation module on top of the classification head to generate rule-specific corrective guidance

Chapter 3

Al Harere & Al Jallad (2023) — Mispronunciation Detection via MFCC + LSTM

Paper Metadata

Title	Mispronunciation Detection of Basic Quranic Recitation Rules using Deep Learnin
Authors	Ahmad Al Harere, Khloud Al Jallad
Institution	Arab International University, Syria
Type	Original model — binary tajweed rule classification (correct / incorrect)
Dataset	QDAT (public)

3.1 Objective

Detect mispronunciation of three specific tajweed rules from audio using deep learning. This is one of the first works to: (a) use a **public dataset** (QDAT) for tajweed mispronunciation, and (b) explicitly model the **temporal/sequential** nature of speech through LSTM rather than treating each frame independently.

3.2 Tajweed Rules Targeted

Rule	Description
Separate Stretching (Madd)	Vowel elongation at end of word when followed by Hamza; duration = 4–5 counts
Tight Noon (Ghunnah)	Nasalisation on aggravated Noon; must hold for 2 counts
Hide (Ikhfa')	Consonant Noon/Tanween pronounced between showing and fading without emphasis, 2 counts

3.3 Dataset — QDAT

QDAT Dataset

Total samples	>1,500 audio clips
Content	Recordings of verse 109, Surah Al-Ma'idah
Labels	Binary per rule: correct / incorrect recitation
Speakers	Both genders, various age groups
Format	WAV, 11 kHz, mono, 16-bit
Split	90% train / 10% test
Access	Public — Kaggle: `annealdahi/quran-recitation`

Key distinction: Unlike all prior works that used private datasets, QDAT is public — enabling direct comparison. The paper is the first to use QDAT.

3.4 Approach & Architecture

3.4.1 Core Idea

Speech is a **time series** — a tajweed error can span multiple consecutive frames. Prior works (SVM, MLP, etc.) treated each frame independently, discarding temporal context. This paper argues LSTM is the correct architecture because it **carries memory across frames**, allowing the model to make decisions based on the full sequence of acoustic events rather than a single snapshot.

3.4.2 Pipeline Overview

Raw WAV $\xrightarrow{\text{Pre-emphasis}}$ Framed signal $\xrightarrow{\text{MFCC}}$ Feature sequence $\xrightarrow{\text{LSTM}}$ Correct / Incorrect

3.4.3 Step 1 — Pre-emphasis

A high-pass filter applied before any feature extraction:

$$H(z) = 1 - \alpha z^{-1}, \quad \alpha = 0.97$$

Purpose: Human speech production naturally attenuates high frequencies. Pre-emphasis boosts them back up, making the signal more balanced and reducing model sensitivity to low-frequency noise. This is a standard preprocessing step often skipped in other works.

3.4.4 Step 2 — MFCC Extraction (detailed pipeline)

The paper gives the most detailed MFCC breakdown of all reviewed works:

Output: A sequence of MFCC vectors — one vector per frame — forming a 2D time-frequency representation of the full audio clip. This sequence (not a single vector) is fed to the LSTM.

3.4.5 Step 3 — LSTM Architecture

Why LSTM over RNN?

Plain RNNs suffer from **vanishing gradients** — information from distant frames fades during backpropagation, making it impossible to learn long-range dependencies. LSTM solves this with a **cell state** C_t that acts as long-term memory, controlled by three gates:

- **Forget gate** $f_t = \sigma(x_t U^f + h_{t-1} W^f)$ — decides what to erase from memory
- **Input gate** $i_t = \sigma(x_t U^i + h_{t-1} W^i)$ — decides what new info to write to memory
- **Output gate** $o_t = \sigma(x_t U^o + h_{t-1} W^o)$ — decides what to output from memory
- **Cell update:** $C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$
- **Hidden state:** $h_t = \tanh(C_t) * o_t$

Why this matters for tajweed: A tajweed rule like Madd (elongation) requires the model to "remember" that a vowel started several frames ago and is still being held — exactly the kind of temporal dependency LSTMs are designed for.

Network structure

```
Input: MFCC sequence (T frames x 40 features)
  -> LSTM(256 units)
  -> LSTM(256 units)
  -> LSTM(256 units)
  -> Dense(16) + ReLU
  -> Dropout(0.2)
  -> Dense(16) + ReLU
  -> Dropout(0.2)
  -> Dense(8)  + ReLU
  -> Dense(1)  + Sigmoid   # binary: correct=1 / incorrect=0
```

Listing 3.1: LSTM model architecture

One separate model trained per tajweed rule (3 models total).
Loss: Binary Cross-Entropy **Optimizer:** RMSprop

3.5 Results

Performance vs. Baselines on QDAT

Model	Sep. Stretching	Hide	Tight Noon
Random Forest	91%	83%	91%
KNN	91%	81%	94%
Logistic Regression	88%	79%	91%
SVM	81%	71%	85%
LSTM (proposed)	**96%**	**95%**	**96%**

F1-scores: 95% / 95% / 97% for the three rules respectively.
LSTM outperforms all baselines on all three rules, with the largest gap on "Hide" (+12% over SVM).

3.6 Key Contributions & Limitations

Contributions	Limitations
First use of QDAT (public) for tajweed MDD	Only 3 tajweed rules covered
Explicit temporal modelling via LSTM	Binary output only (no rule explanation or error type)
Detailed MFCC pipeline with pre-emphasis	Small dataset (1,500 clips); single verse only
Outperforms all classical ML baselines	No feedback generation — detection only

3.7 Relevance to Your Research

Key Takeaways

- **Architecture direction:** LSTM (or its successors: GRU, Transformer) should be preferred over frame-independent classifiers for tajweed because rules are inherently time-dependent
- **Pre-emphasis matters:** often skipped in other works but improves high-frequency sensitivity — relevant for detecting subtle phonetic distinctions in tajweed
- **QDAT dataset:** publicly available and already used as a benchmark — your model should report results on it for comparability

- **Open problem:** the paper detects errors but produces no corrective feedback — this is the gap your system should address
- **Scale limitation:** 3 rules, 1 verse, 1,500 clips — a production system needs to cover the full tajweed rule set across diverse recitations

Chapter 4

Al-Ayyoub et al. (2018) — CDBN + SVM for Eight Tajweed Rules

Paper Metadata	
Title	Using Deep Learning for Automatically Determining Correct Application of Basic Quranic Recitation Rules
Authors	Mahmoud Al-Ayyoub, Nour Alhuda Damer, Ismail Hmeidi
Institution	Jordan University of Science and Technology, Jordan
Journal	International Arab Journal of Information Technology, Vol. 15, No. 3A, 2018, pp. 620–625
Type	Original model — multi-class tajweed classification (16 classes)
DOI	`iajit.org/PDF/Special Issue 2018, No. 3A/17417.pdf`

Open Science Status	
Code	**Available** — GitHub: `github.com/malayyoub/Ahkam-Al-Tajweed` Repository published by first author (Mahmoud Al-Ayyoub). Contains feature extraction scripts and classifier code.
Dataset	**Partially available** — Raw audio data linked via ResearchGate (`researchgate.net/publication/327869102`). Dataset is in-house but shared through ResearchGate on request.

4.1 Objective

Build a system capable of identifying which of eight tajweed rules is present in an audio clip of Quranic recitation, and whether that rule is applied correctly or incorrectly. The classification problem is therefore framed as **16 classes** (8 rules × {correct, incorrect}). Unlike all prior works that were restricted to a small number of verses or chapters, this paper targets the **entire Holy Quran** — covering every occurrence of the eight rules

across all 114 Surahs.

4.2 Dataset

Table 4.1: In-house dataset composition

Property	Value
Total audio files	3,071 WAV clips
Reciters	10 expert reciters (5 male, 5 female)
Labels	16 classes: rule identity + correct/incorrect
Coverage	All occurrences in the entire Holy Quran
Availability	Private (in-house); raw data shared on ResearchGate

Table 4.2: Tajweed rules targeted and sample distribution (selected rules)

Rule	Description	Correct	Incorrect
EdgamMeem	Merging through Meem with Ghunnah	120	30
EkhfaaMeem	Concealment of Meem Sakinah before Ba	120	30
Tafkheem Lam	Heavy Lam in Allah (Fatha/Dhamma)	600	462
Tarqeeq Lam	Light Lam in Allah (Kasrah)	255	164
Edgam Noon (Noon)	Assimilation into Noon	276	68
Edgam Noon (Meem)	Assimilation into Meem	213	53
Edgam Noon (Waw)	Assimilation into Waw	104	26
Edgam Noon (Ya')	Assimilation into Ya'	440	110

Class imbalance note: Tafkheem Lam has over 1,000 samples while Edgam Noon (Waw) has only 130. This reflects the natural occurrence frequency in the Quran but introduces a class imbalance challenge that the paper does not explicitly address.

4.3 Approach & Architecture

The core contribution is the combination of traditional hand-crafted features with unsupervised deep features from a Convolutional Deep Belief Network (CDBN), with final classification performed by a classical SVM.

4.3.1 Traditional Feature Extraction (hand-crafted)

Four techniques are applied in parallel and concatenated:

Feature ablation finding: LPC was eliminated through systematic ablation — removing it had zero impact on accuracy because LPC does not use overlapping windowed analysis (each pth-order depends only on the $(p-1)$th sample), making it redundant given MFCC. The optimal traditional set is **MFCC + WPD + HMM-SPL** (614 features total).

Table 4.3: Traditional feature extraction techniques

Technique	Features	What it captures
MFCC	100	Spectral envelope, vocal tract shape, phoneme identity
LPC	—	Vocal tract resonances (later removed via ablation)
WPD	508	Multi-resolution time-frequency; transient patterns
HMM-SPL	6	Spectral peak locations (formant-like, compact)

4.3.2 Non-Traditional Feature Extraction: CDBN

A **Convolutional Deep Belief Network (CDBN)** is used for unsupervised feature learning from audio spectrograms. CDBN is built from stacked Convolutional Restricted Boltzmann Machines (CRBMs) and predates modern SSL models (wav2vec 2.0, HuBERT). The pipeline is:

```
Step 1: Convert audio clip to spectrogram
        (20 ms window, 10 ms overlap)

Step 2: PCA whitening
        (80 components -- reduces dimensionality, decorrelates)

Step 3: Train Layer 1 CRBM
        (300 bases, filter length=6, max-pooling ratio=3)
        -- learns local spectral patterns from unlabelled
           spectrograms

Step 4: Train Layer 2 CRBM
        (300 bases, trained on Layer 1 activations)
        -- learns higher-level combinations of Layer 1 patterns

Step 5: Extract mean + SD of activations per layer
        --> 600 features per layer x 2 layers = 1,200 features
```

Listing 4.1: CDBN feature extraction pipeline

CDBN learns data-driven representations not available in the fixed Mel filterbank of MFCC — capturing temporal patterns and multi-scale spectral structure. It is now superseded by fine-tuning pre-trained SSL models such as wav2vec 2.0 or HuBERT.

4.3.3 Classification: SVM on Fused Features

All features are concatenated and fed to a Support Vector Machine (SVM):

Extraction time: **15.2 seconds per audio file** — a significant inference cost. Multiple classifiers were evaluated (KNN, RF, ANN/MLP, Bagging, Multiclass RF); SVM consistently outperformed all others on this task.

Table 4.4: Final feature vector composition

Feature source	**# Features**
MFCC	100
WPD	508
HMM-SPL	6
CDBN (L1 + L2)	1,200
Total	**1,814**

4.3.4 Evaluation Protocol

Two evaluation modes are reported:

- **10-fold cross-validation** on the full 3,071-sample dataset.
- **Generalisation test**: model trained on 70% of the data from two reciters, tested on their remaining 30% (525 instances from new, unseen verses not in the training pool). This is a more realistic evaluation than standard cross-validation.

4.4 Limitations

- **Private dataset**: although raw data is shared on ResearchGate, it is not a standardised public benchmark and cannot be compared against by future works without access.
- **Computational cost**: 15.2 seconds per audio file for 1,814 features is impractical for real-time or mobile applications.
- **CDBN is outdated**: the unsupervised pre-training approach is now far superseded by SSL models (wav2vec 2.0, HuBERT, Whisper encoder) which learn richer representations from much larger unlabelled corpora.
- **Class imbalance unaddressed**: the significant sample imbalance across the 16 classes is not handled in the methodology.
- **No feedback generation**: outputs a class label only — no corrective guidance provided to the learner.
- **2018 approach**: no attention mechanisms, no end-to-end learning, no transformer-based models.

4.5 Relevance to Your Research

Key Takeaways

- **Widest rule coverage so far**: 8 rules, both correct and incorrect, entire Quran — this scope should inform your target dataset coverage.
- **Feature combination matters**: MFCC alone (∼94%) is significantly weaker than MFCC + WPD + CDBN (97.8%) — complementary features improve classification.
- **WPD is worth considering**: it captures transient multi-resolution patterns complementary to MFCC's spectral envelope and is underused in more recent works.
- **Replace CDBN with SSL**: use pre-trained encoders (wav2vec 2.0, HuBERT, Whisper) as the modern equivalent of CDBN-style unsupervised feature learning.
- **SVM as a strong baseline**: with rich hand-crafted features SVM is competitive; end-to-end fine-tuning of a Transformer encoder is the modern equivalent and will be stronger.
- **Generalisation test matters**: evaluation on new unseen verses is a more realistic protocol than same-verse cross-validation — your evaluation should follow this.
- **Code is public**: the GitHub repository (`github.com/malayyoub/Ahkam-Al-Tajweed`) provides a reusable starting point for feature engineering baselines.

Chapter 5

Salim et al. (2025) — Dataset Split Experiment for Tajweed Classification

Paper Metadata	
Title	Dataset Ratio Experiment for Tajweed Law Recognition Using MFCC and CNN Features
Authors	Mas Muhammad Aqil Salim, Ani Dijah Rahajoe, Anggraini Puspita Sari
Institution	UPN Veteran Jawa Timur, Indonesia
Journal	JEEMECS, Vol. 8, No. 2, August 2025, pp. 91–97
Type	Original model — multi-class tajweed classification + dataset split study
ISSN	2614-4859

Open Science Status	
Code	**Not available** — No GitHub repository found. No supplementary material or code link in the paper.
Dataset	**Not available** — Dataset is private (in-house). Collected from direct recordings and YouTube observations; not released publicly. No request mechanism indicated.

5.1 Objective

Classify six Nun Sukun/Tanwin tajweed rules from audio using MFCC + CNN, and specifically investigate **how the train/val/test split ratio affects classification performance**. The primary research question is not the architecture itself, but the data configuration around it — making this one of the few papers in the field to treat dataset partitioning as a first-class experimental variable.

5.2 Dataset

Table 5.1: Dataset summary

Property	Value
Total samples	1,344 audio clips
Sources	Direct recordings + YouTube observations
Classes	6 (see Table 5.2)
Clip length	Standardised to 2 seconds (trimmed or padded)
Preprocessing	Silence removal, noise reduction
Features	MFCC with 40 coefficients
Availability	Private — not released

Table 5.2: Tajweed classes targeted (Hukum Nun Sukun and Tanwin family)

Class	Description
Idghom Bighunnah	Assimilation of Noon/Tanween into next letter with nasalisation
Idghom Bilaghunnah	Assimilation without nasalisation
Idzhar Halqi	Clear, unmerged pronunciation from throat letters
Ikhfa Haqiqi	Partial concealment with nasalisation
Iqlab	Conversion of Noon/Tanween into Meem-like sound
No Class	Neutral reference class (no rule active)

5.3 Approach & Architecture

5.3.1 Pipeline

```
Raw Audio
  --> Silence removal + Noise filter
  --> Clean clip (padded/trimmed to 2 seconds)
  --> MFCC extraction (40 coefficients, Hamming window, FFT, DCT
     )
  --> 2D MFCC feature map
  --> CNN
  --> 6-class Softmax output
```

Listing 5.1: Full processing pipeline

5.3.2 CNN Architecture

Three convolutional layers with progressively increasing filter counts, using Leaky ReLU throughout (instead of standard ReLU) and Batch Normalisation:

```
Input: MFCC feature map (40 coefficients x frames)

Layer 1: Conv2D (32 filters, 4x4, padding='same')
         + LeakyReLU + BatchNorm
         + MaxPooling2D (2x2)

Layer 2: Conv2D (48 filters, 4x10, padding='same')
         + LeakyReLU + BatchNorm
         + AveragePooling2D        <-- note: Average, not Max

Layer 3: Conv2D (64 filters, 4x10, padding='same')
         + LeakyReLU + BatchNorm
         + MaxPooling2D

--> GlobalAveragePooling  (flattens all feature maps to 1D)
--> BatchNorm
--> Dense (64) + LeakyReLU + Dropout (0.4)
--> Dense (6)  + Softmax    (6 tajweed classes)
```

Listing 5.2: CNN architecture for tajweed classification

Why Leaky ReLU? Standard ReLU sets negative activations to zero, causing "dead neurons" that never recover gradients. Leaky ReLU allows a small non-zero gradient for negatives ($f(x) = \max(\alpha x, x),\ \alpha \ll 1$), keeping all neurons active throughout training.

Why AveragePooling in Layer 2? MaxPooling keeps only the strongest activation (dominant local peak). AveragePooling computes the spatial mean — more suitable for capturing broad spectral patterns rather than sharp local peaks. The alternating pooling strategy (Max/Average/Max) provides complementary spatial summaries at each scale.

5.3.3 Key Experiment — Dataset Split Ratios

All model parameters were held constant; only the data split varied across three scenarios:

Table 5.3: Dataset split comparison (all other hyperparameters fixed)

Scenario	Split	Accuracy	Macro F1	Loss	Test samples
A (best)	80:10:10	89%	0.87	0.5061	144
B	70:15:15	87%	0.84	0.5103	208
C (worst)	60:20:20	80%	0.80	0.5168	280

Key finding: More test data degraded performance. A larger training set (Scenario A) allowed the CNN to learn more representative features per class. Scenario C's

larger test set introduced higher acoustic diversity that a less-trained model could not generalise to.

Per-class pattern: Acoustically distinct classes (Iqlab, No Class) achieved F1 > 0.97 across all scenarios. Acoustically similar classes (Idghom Bighunnah, Idzhar Halqi) were the most sensitive to training set size, dropping significantly in Scenario C.

Fixed training hyperparameters across all scenarios: 40 MFCC coefficients, max input length 400 frames ($\approx$2 seconds), batch size 8, 100 pre-training epochs, 50 fine-tuning epochs.

5.4 Limitations

- **Private, small dataset**: 1,344 samples from a single source with no cross-dataset validation; results may not generalise.
- **Only Nun Sukun/Tanwin rules**: no other tajweed rule families covered.
- **No temporal modelling**: CNN treats the MFCC as a static 2D image, ignoring frame-to-frame sequence dependencies. Rules with inherent temporal structure (elongation, nasalisation duration) are better handled by LSTM or Transformer architectures.
- **No feedback generation**: classification only; no corrective output.
- **No code or data released**: results are not reproducible without contacting the authors.

5.5 Relevance to Your Research

Key Takeaways

- **Practical data split guidance**: for datasets of ∼1,000–2,000 samples, an 80:10:10 split is empirically better than more conservative splits — maximise the training portion.
- **Leaky ReLU** is a simple drop-in improvement over standard ReLU worth adopting in all CNN layers.
- **Mixed pooling** (MaxPooling + AveragePooling in different layers) captures both local peaks and broad spectral patterns — a practical architectural choice.
- **CNN vs. LSTM trade-off**: CNN is simpler and faster but ignores temporal order. For time-dependent rules (elongation, nasalisation duration), LSTM is better motivated. CNN works well for rules with strong spectral signatures (articulation-based classes like Iqlab).
- **Class difficulty insight**: acoustically similar classes (e.g. Idghom Bighunnah vs. Ikhfa Haqiqi) will be the hardest to separate — your dataset and architecture should account for these confusable pairs.

Chapter 6

Alrashoudi et al. (2025) — Arabic MDD via Transformer Fine-Tuning

Paper Metadata

Title	Improving mispronunciation detection and diagnosis for non-native learners of the Arabic language
Authors	Norah Alrashoudi, Hend Al-Khalifa, Yousef Alotaibi
Journal	Discover Computing, Vol. 28, Article 1 (2025)
Type	Original model — Arabic phoneme-level MDD
DOI	10.1007/s10791-024-09489-8
Dataset	L2-KSU (available on GitHub upon request)

Scope Note

This paper targets **general Arabic phoneme MDD** for non-native speakers, not tajweed rules specifically. Its direct value to your project is: (1) it is the most technically advanced Arabic-language MDD system in this survey, using Transformer fine-tuning on Arabic speech including Quranic text; (2) it introduces and releases the **L2-KSU dataset**, the only public Arabic MDD corpus found across all reviewed papers; and (3) it provides a fully documented fine-tuning protocol for Wav2Vec2.0, HuBERT, and Whisper on Arabic — directly reusable for a tajweed system.

6.1 Objective

Build and benchmark an Arabic MDD framework for non-native speakers that: (1) detects mispronounced phonemes in continuous Arabic speech, and (2) diagnoses the type of each error — insertion, deletion, or substitution — at the phoneme level. Additionally, the paper constructs and releases a new Arabic speech dataset (L2-KSU),

conducts a human perceptual test on 200 utterances, and compares automatic vs. human detection/diagnosis accuracy.

6.2 Dataset — L2-KSU

Property	Value
Total utterances	4,086 WAV files
Total duration	6 h 6 min
Speakers	40 native Arabic + 40 non-native (West/Central Africa)
Gender split	Equal male/female per speaker type
Sampling rate	16 kHz mono
Content	Quranic text + MSA sentences
Annotation	Canonical text + real transcription with error labels
Error labels	Insertion / Deletion / Substitution per phoneme
Train split	60 speakers (native + non-native), 3,273 utterances
Test split	20 speakers (non-native only), 813 utterances
Availability	Public — GitHub: `github.com/nrshoudi/L2-KSU-Dataset`

Key design decision: native speakers appear only in training; the test set contains only non-native speakers. This ensures the system is evaluated on unseen pronunciation error patterns from unseen speakers — a more realistic and harder protocol than cross-validation.

Sentence selection: utterances were chosen to maximise exposure to pharyngeal and emphatic consonants (/Q/, /H/, /d/, etc.) that are hardest for non-native speakers. Several Quranic sentences (Al-Fatiha, Al-Falaq, etc.) are included.

6.3 Methodology Overview

The framework has two sequential phases identical across all three model families:

```
Phase 1 -- ASR:
  Raw audio (16 kHz WAV)
  --> Preprocessing (noise removal via PRAAT, mono conversion)
  --> Model-specific feature extraction
  --> Phoneme-sequence transcription (Ph1, Ph2, ..., Phn)

Phase 2 -- MDD:
  Predicted transcription
  --> Levenshtein distance vs. canonical text
  --> Correctly pronounced / Mispronounced
  --> Error type analysis: Insertion / Deletion / Substitution
```

Listing 6.1: Two-phase MDD framework (all models)

The MDD diagnosis step is **not learned** — it is computed deterministically via Levenshtein distance alignment between the ASR output and the canonical phoneme sequence. The deep learning model is responsible only for phoneme recognition (Phase 1); the error type classification follows from string alignment (Phase 2).

6.3.1 Model 1 — Wav2Vec2.0 (XLS-R-300M)

```
Raw waveform (16 kHz float array)
--> CNN feature encoder (local acoustic features)
--> Transformer context network (contextualized representations)
--> CTC decoder --> phoneme-based transcription
```

Listing 6.2: Wav2Vec2.0 / HuBERT pipeline

The multilingual model `Wav2Vec2-XLS-R-300M` was used: pre-trained on 436k hours of unlabelled speech across 128 languages including Arabic, with 300M parameters and 24 Transformer layers (width 1024, 16 heads). Fine-tuned end-to-end with CTC loss on L2-KSU. Training parameters: batch size 2, lr = 1e-4, 20 epochs, warm-up ratio 0.1. Full 315M parameters trained (no PEFT applied).

6.3.2 Model 2 — HuBERT (Large + XLarge)

HuBERT follows the same CNN + Transformer + CTC architecture as Wav2Vec2.0. Pre-trained via masked prediction of k-means cluster assignments of MFCC frames (offline targets), on 960h (base) / 60kh (large) of English Librispeech. Three variants fine-tuned: HuBERT-large (315M), HuBERT-large with PEFT (reduced to 3.1M trainable params), HuBERT-xlarge with PEFT (reduced to 7.8M).

PEFT techniques applied to HuBERT-xlarge:

- **LLM.int8**: reduces floating-point precision to 8-bit integer matrix multiplication, cutting memory footprint of stored weights.
- **LoRA (Low-Rank Adaptation)**: freezes all pre-trained weights; inserts trainable rank-decomposition matrices $W = W_0 + BA$ (where $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$, $r \ll \min(d, k)$) into every Transformer layer. Only BA is updated during fine-tuning.

Critical finding: PEFT with LoRA + int8 reduced HuBERT-large from 315M to 3.1M trainable parameters, but catastrophically degraded PER from 3.6% to 79.0%. PEFT is not suitable for low-resource Arabic fine-tuning at this compression ratio.

6.3.3 Model 3 — Whisper

```
Raw audio
--> Log-Mel spectrogram (80 Mel bins for v2, 128 for v3)
    segmented into 30-second windows
```

```
--> 2x Conv1D + GELU (subsampling encoder frontend)
--> Transformer Encoder (sinusoidal positional encoding)
    --> encoder hidden states
--> Transformer Decoder (learned positional encoding,
    cross-attention to encoder states)
    --> autoregressive token prediction
--> Phoneme-based transcription
```

Listing 6.3: Whisper pipeline

Six Whisper variants fine-tuned: tiny (39M), base (74M), small (244M), medium (769M), large-v2 (1550M), large-v3 (1550M). PEFT applied to all Whisper variants, training only 1–1.5% of parameters (e.g. large-v2: 1550M → 15.7M trainable). Training parameters: batch size 6, lr = 1e-3, 10 epochs, 50 warm-up steps. Whisper large-v3 differs from large-v2 in: 128 Mel bins (vs. 80), trained on 5M hours (1M weakly-labelled + 4M pseudo-labelled) vs. 680k hours.

6.4 Results

6.4.1 Phoneme Recognition (PER)

Model	Variant	PER (%)
CNN-RNN-CTC (baseline)	—	13.3
Wav2Vec2.0	XLS-R	3.2
HuBERT	Large (full)	3.6
HuBERT + PEFT	Large	79.0
HuBERT + PEFT	XLarge	40.0
Whisper	Tiny	3.5
Whisper	Base	3.9
Whisper	Small	3.2
Whisper	**Medium**	**3.12**
Whisper	Large-v2	3.18
Whisper	Large-v3	3.6

6.4.2 MDD Performance

Key finding: Whisper-small achieves the best MDD performance despite having far fewer parameters than large variants. Bigger is not always better — the sweet spot for this dataset is Whisper-small, where training data fit and generalisation are optimal.

6.4.3 Human Perceptual Test vs. Automatic Verification

The automatic system surpasses human raters on both metrics. Inter-rater agreement (Fleiss' kappa) among the four native Arabic reviewers was only 50.9% (moderate) —

Model	Prec.	Rec.	F1	FRR	FAR	DER	Diag.	Det.
CNN-RNN-CTC	84.1	76.2	80.0	31.4	23.7	44.5	55.4	73.8
Wav2Vec2 XLS-R	92.8	87.3	90.0	12.0	12.6	24.3	75.6	87.5
HuBERT-large	94.9	86.9	90.8	8.2	13.0	21.8	78.1	88.6
Whisper-tiny	94.1	86.5	90.2	11.8	13.4	26.5	73.4	87.0
Whisper-base	94.8	89.6	92.1	10.6	10.3	23.3	76.6	89.5
Whisper-small	**96.2**	**91.0**	**93.5**	**7.8**	**8.9**	**19.9**	**80.0**	**91.3**
Whisper-medium	94.1	89.8	91.9	12.2	10.1	24.3	75.6	89.1
Whisper-large-v2	96.5	89.9	93.1	7.0	10.0	20.4	79.5	90.8
Whisper-large-v3	96.8	89.4	93.0	6.2	10.5	21.5	79.6	90.7

Table 6.1: All values in %. Best MDD model: Whisper-small (Detection 91.3%, Diagnosis 80.0%). Best PER model: Whisper-medium (3.12%).

	Diagnosis Accuracy	Detection Accuracy
Human (4 native reviewers)	66.0%	90.5%
Whisper-small (automatic)	80.4%	97.0%

confirming that even native speakers disagree on Arabic pronunciation errors, especially for acoustically similar phoneme pairs (/ṣ/ vs. /s/, /ð/ vs. /ð/).

6.4.4 Error Type Distribution (Whisper-small on test set)

Out of 1,750 detected pronunciation errors from 813 non-native utterances: substitution errors were the most frequent (957), followed by deletion (706) and insertion (87). The dominant substitution pairs were pharyngeal replacements (/Q/ → /P/ and /H/ → /h/), emphatic consonant confusion (/d/ → /d/), and interdental substitutions (/ð/ → /z/).

6.5 Limitations

- **No tajweed rules:** error taxonomy is insertion/deletion/substitution at the phoneme level only. Tajweed rules (Madd duration, Ghunnah nasalisation, Idgham assimilation) are not modelled or labelled.
- **Levenshtein-based diagnosis is brittle:** the diagnosis step is a string-alignment heuristic, not a learned model. It cannot distinguish tajweed rule violations from general phoneme errors.
- **Non-native population is geographically narrow:** all non-native speakers are from West and Central Africa (Wolof/Yoruba L1). Results may not generalise to other L1 backgrounds (Asian, European, etc.).
- **PEFT degrades HuBERT severely:** aggressive parameter reduction (315M → 3.1M) destroys performance, suggesting LoRA rank was too low for Arabic phoneme diversity. This is a practical warning for resource-constrained fine-tuning.

- **No corrective feedback:** the system outputs error type labels only. No guidance is given on how to correct the detected error.
- **Short vowel annotation gap:** short vowels (/a/, /i/, /u/) are not annotated in the dataset, causing the model to confuse them with long vowels in the confusion matrix.

6.6 Relevance to Your Research

Key Takeaways

- **Whisper is the best model for Arabic phoneme recognition among SSL models:** Whisper-small achieves 3.2% PER and 91.3% detection accuracy on Arabic, surpassing Wav2Vec2.0 and HuBERT in MDD despite smaller parameter count. For your tajweed system, fine-tuning Whisper-small on a tajweed-labelled corpus is the highest-priority architecture to try.
- **Use Whisper's encoder-decoder for tajweed sequence modelling:** Whisper's decoder autoregressively generates output conditioned on the full encoder context and preceding tokens. For tajweed, the decoder could be conditioned on the canonical phoneme sequence and trained to output tajweed rule violation labels per position.
- **The L2-KSU dataset contains Quranic text and is publicly available:** it is the only public Arabic MDD corpus identified in this entire survey. Its Quranic sentences (Al-Fatiha, Al-Falaq, dhikr phrases) can serve as a baseline corpus for pre-training or transfer learning before fine-tuning on tajweed-specific data.
- **Levenshtein distance is sufficient for error-type diagnosis when ASR quality is high:** given a PER of 3.1%, string alignment reliably identifies insertion/deletion/substitution errors. For tajweed, a similar approach can assign error types at the phoneme level; the rule-specific feedback layer must then be added on top.
- **Do not use PEFT/LoRA aggressively on small Arabic datasets:** the catastrophic failure of HuBERT + PEFT (PER 3.6% $\rightarrow$ 79%) shows that reducing trainable parameters too far when the target domain is linguistically complex destroys transfer. Use full fine-tuning or conservative LoRA ranks ($r \geq 16$).
- **Human raters disagree on Arabic pronunciation errors at 50.9% kappa:** this validates using automatic verification as the ground truth for tajweed annotation, and implies that your tajweed dataset annotation should be done by trained Quranic scholars (huffaz) rather than general Arabic speakers.
- **Substitution errors dominate over deletion and insertion** (55% of all errors): for tajweed, the equivalent dominant error type will likely be rule-substitution (e.g. applying Izhaar instead of Idgham). Design your classifier and evaluation metrics accordingly.

Chapter 7

Xie et al. (2026) — LLM-Based MDD with Articulatory Feedback

Paper Metadata

Title	Mispronunciation detection and diagnosis based on large language models
Authors	Yanlu Xie, Huihang Zhong, Xuhui Lan, Wenwei Dong
Journal	Computer Speech & Language, Vol. 99 (2026) 101942
Type	Original model — LLM-based MDD + feedback generation
DOI	10.1016/j.csl.2026.101942
Language	L2 English + L2 Chinese (not Arabic)
Dataset	L2-ARCTIC (English), 103 Non-Native Chinese Dataset

Scope Note

This paper is neither Arabic nor Quran-specific. Its relevance to your project is architectural and methodological: (1) it introduces the most advanced feedback generation pipeline found in this survey — moving beyond detection to produce **actionable, articulatory-level corrective guidance**; (2) it proposes a two-framework architecture (end-to-end multimodal LLM vs. cascaded detection + LLM) directly applicable to tajweed feedback design; (3) it introduces the Viterbi algorithm as a superior alternative to Levenshtein distance for phoneme-level error localization; and (4) it proposes two novel evaluation metrics (G-Score, A-Score) better suited to feedback quality assessment than standard ASR metrics.

7.1 Objectives

This paper addresses two critical gaps in existing MDD systems:

1. **Feedback poverty:** most MDD systems output binary labels (correct/incorrect)

or error type tags (substitution/deletion/insertion) without explaining why the error occurred or how to physically correct it.

2. **Data dependency:** conventional MDD requires task-specific annotated corpora and ASR-like training for each target language/accent. LLMs can partially bypass this through prompt engineering and fine-tuning.

Two frameworks are proposed and compared:

- **Framework 1 (End-to-End):** a multimodal speech-text LLM (Qwen2-Audio) that takes raw L2 speech directly and generates textual feedback end-to-end.
- **Framework 2 (Cascaded):** a dedicated MDD acoustic model (Fu et al. 2021, CNN-RNN-CTC) detects errors; then a text LLM (GPT-4, ChatGLM-6B, etc.) generates articulatory feedback conditioned on the detected phoneme errors.

7.2 Framework 1 — End-to-End Multimodal LLM (Qwen2-Audio)

7.2.1 Architecture

```
L2 learner speech (raw waveform)
--> Log-Mel spectrogram
--> Qwen2AudioEncoder:
      2x Conv1D (subsampling)
    + Positional Embedding Layer
    + 32x Qwen2AudioEncoderLayers
      (Self-Attention + LayerNorm + FFN + Residual)
    + AvgPool1d
--> Qwen2AudioMultiModalProjector:
      Single Linear layer (dim: 1280 -> 3584)
    [audio embedding aligned to text embedding space]
--> Concatenate(audio_embedding, text_prompt_embedding)
--> 32x Qwen2DecoderLayers
    (Self-Attention + Rotary Embeddings + Qwen2MLP)
--> Output: Vocabulary Space (156,032 tokens)
--> Textual feedback / phoneme sequence
```

Listing 7.1: Qwen2-Audio end-to-end MDD pipeline

The audio encoder is initialized from `whisper-large-v3` (1280-dim, 32 layers). The key modification in this paper is **replacing the original encoder with a version additionally fine-tuned on L2 speech data** (`Belle-whisper-large-v3-turbo-zh`), which was itself pre-trained on 24%+ more Chinese ASR data (AISHELL-1/2, WenetSpeech, HKUST).

7.2.2 Training Protocol

Two-stage fine-tuning with LoRA (rank = 8):

1. **Stage 1:** Full fine-tuning of Belle-whisper-large-v3-turbo-zh on L2 speech data (L2-ARCTIC + 103 Dataset). Hardware: 1×H800 GPU, batch size = 8, lr = 0.001, 10 epochs.
2. **Stage 2:** Fine-tune entire Qwen2-Audio model (with swapped encoder from Stage 1) on the Non-Native Speech Instruction Dataset (MDD + feedback tasks jointly). Hardware: 1×H800 GPU, batch size = 32, lr = 0.0001, 10 epochs, gradient accumulation = 16.

7.2.3 Prompt Engineering Experiments

Three prompt augmentation strategies were tested:

1. Baseline: no additional information.
2. + Reading text: the canonical text the learner was reading.
3. + Correct phoneme sequence: the reference phoneme sequence as prompt.

Critical negative result: providing the correct phoneme sequence as a prompt caused Precision to rise from 0.49 to 0.87, but Recall collapsed from 0.63 to 0.21, and F1 fell from 0.55 to 0.33. The model replicated the reference phonemes verbatim instead of detecting deviations. This is a fundamental prompt injection failure and a key practical warning for LLM-based MDD design.

7.2.4 English MDD Results (L2-ARCTIC)

Model	Recall	Precision	F1
Baseline1 (Fu et al. 2021, CNN-RNN-CTC)	0.5612	0.5604	0.5608
Baseline2 (Peng et al. 2022, Text-Aware)	0.5218	0.8718	0.6175
Whisper-en (fine-tuned on L2)	0.5887	0.5370	0.5616
Qwen2-audio-ori-en (original encoder)	0.7162	0.3912	0.5060
Qwen2-audio-sub-en (swapped encoder)	0.6327	0.4864	0.5500
+ reading text prompt	0.6341	0.4877	0.5514
+ correct phoneme sequence	0.2051	0.8730	0.3321

Table 7.1: Swapping the encoder improves F1 by +4.4 points over original Qwen2-Audio. Reading text prompt gives marginal gain (+0.14). Correct phoneme prompt catastrophically reduces recall.

7.2.5 Chinese MDD Results (103 Dataset)

Key finding: Adding speakers from more L1 backgrounds progressively *degrades* performance (Whisper-zh-jp: F1=0.47 → Whisper-zh-jp-cn: 0.43 → Whisper-zh-jp-cn-kp:

Setting	Model	F1	Note
Single-L1 (Japanese)	Whisper-zh-jp	0.4701	—
	Qwen2-audio-sub-zh-jp	0.4752	Qwen2 > Whisper
	+ prior L1 knowledge prompt	0.4803	+1.02%
Multi-L1 (all nationalities)	Whisper-zh	0.3416	Low generalization
	Qwen2-audio-sub-zh	0.4377	Encoder swap helps
	+ L1 country info prompt	0.4545	+1.68%

0.41). Mixed L1 training introduces phonetic interference. Explicitly providing L1 country information in the prompt partially mitigates this.

7.3 Framework 2 — Cascaded: Acoustic MDD + LLM Feedback

7.3.1 Pipeline

```
L2 speech (raw waveform)
--> MDD acoustic model (Fu et al. 2021, CNN-RNN-CTC)
--> Predicted phoneme sequence (Ph1, Ph2, ..., PhN)

Viterbi alignment:
  Given: predicted sequence O = {o1, ..., oT}
         reference sequence S = {s1, ..., sN}
  Compute: P(Q|O) = prod_t P(q_t | q_{t-1}) * P(o_t | q_t)
  Backtrack: optimal state sequence Q* identifying mismatch
     positions

--> Mispronounced phoneme positions identified

Articulatory feature mapping:
  Each phoneme --> 8-dimensional vector:
    [jaw, lip_separation, lip_rounding, tongue_frontness,
     tongue_height, tongue_tip, velum, voicing]

  Example: 's' = [1,2,2,3,3,3,0,0]
           'z' = [1,2,2,3,3,3,0,1]
           Difference: dim 8 (voicing) only
                       --> "vocal cord vibration" feedback

--> LLM (GPT-4 or fine-tuned ChatGLM-6B)
    receives: (error phoneme, reference phoneme,
               articulatory feature diff)
    generates: articulatory corrective feedback text
```

Listing 7.2: Viterbi-based articulatory feedback pipeline

7.3.2 Why Viterbi over Levenshtein?

Levenshtein distance assigns equal cost to all substitution errors regardless of phonetic similarity. For example, substituting /s/ → /z/ (voicing only) costs the same as /s/ → /m/ (completely different articulation). Viterbi uses transition probabilities $P(q_t|q_{t-1})$ that encode phonetic neighbourhood relationships, making the error localization phonetically informed and enabling more precise feedback.

7.3.3 Articulatory Feature Space

Feature	Classes	Cardinality
Jaw	Nearly Closed / Neutral / Slightly Lowered / Lowered	4
Lip separation	Closed / Slightly Apart / Apart / Wide Apart	4
Lip rounding	Rounded / Slightly Rounded / Neutral / Spread	4
Tongue frontness	Back / Slightly Back / Neutral / Slightly Front / Front	5
Tongue height	Low / Mid / Mid-High / High	4
Tongue tip	Low / Neutral / Dental / Nearly Alveolar / Alveolar	5
Velum	Closed / Open	2
Voicing	Unvoiced / Voiced	2

Table 7.2: 8-dimensional articulatory feature vector (Tepperman & Narayanan 2007). Each phoneme is represented as a vector over these dimensions. Feedback is generated by comparing the canonical and mispronounced phoneme vectors.

7.3.4 Fine-Tuning LLMs with Splitting Process Datasets (SPD)

Direct end-to-end fine-tuning of ChatGLM-6B on the feedback task failed. The proposed solution decomposes training into two sequential datasets:

1. **Problem Dataset:** trains the model to identify and align phoneme-level errors (input: predicted + reference sequences; output: error positions and types).
2. **Answer Dataset:** trains the model to generate articulatory feedback (input: error positions from Problem Dataset output; output: corrective text using articulatory feature language).

During inference, the model first generates Problem output (error localization), then uses that as input to generate the Answer (feedback text). SPD significantly outperforms direct end-to-end fine-tuning by enforcing a chain-of-thought reasoning structure.

7.3.5 Feedback Results (Subjective Evaluation — MOS)

Method	Comprehensibility	Helpfulness
GPT-4 (baseline, no Viterbi, no AF)	3.21	3.00
GPT-4 + Viterbi alignment	3.50	3.30
GPT-4 + Viterbi + Articulatory Features	3.46	**3.79**

Table 7.3: Mean Opinion Scores from 10 L2 English learners (5-point Likert scale). Articulatory features significantly improve helpfulness (+0.79 over baseline) at a slight cost to comprehensibility (−0.04 vs. Viterbi-only), because phonetic terminology ("tongue tip", "lip rounding") is unfamiliar to learners.

7.4 Evaluation Metrics

7.4.1 G-Score (existing)

A composite metric for MDD feedback quality:

$$\text{G-Score} = 0.2 \times \text{BLEU-4} + 0.25 \times \text{ROUGE-2} + 0.25 \times \text{CHRF} + 0.3 \times \text{Semantic Similarity}$$

7.4.2 A-Score (proposed)

BLEU-4 is removed (poor correlation with feedback utility in L2 settings) and replaced with a **domain terminology richness** count:

$$\text{A-Score} = 0.2 \times \text{ROUGE-2} + 0.2 \times \text{CHRF} + 0.3 \times \text{Semantic Similarity} + 0.3 \times \left(\frac{\texttt{detail\_richness}}{5} \right)$$

where `detail_richness` counts occurrences of articulatory vocabulary ("tongue", "lip", "velum", "alveolar", "voicing", etc.) in the generated feedback, normalized by 5.

Validation: A-Score produces statistically more significant differences between feedback with and without articulatory analysis than G-Score across all four LLMs tested (t-test $p < 0.01$ in all cases).

Model	as-no	as	gs-no	gs
ChatGLM3-6B	0.29	0.50	0.26	0.40
Yi-6B	0.31	0.47	0.28	0.39
LLaMA-8B	0.30	0.48	0.27	0.38
Qwen-7B	0.28	0.45	0.25	0.35

Table 7.4: as = A-Score with articulatory analysis, as-no = without. A-Score gap (≈0.2) is consistently larger than G-Score gap (≈0.1), confirming A-Score's higher sensitivity.

7.5 Limitations

- **Not Arabic or Quran-specific:** experiments are on English (L2-ARCTIC) and Mandarin (103 Dataset). All articulatory feature mappings are for English/Mandarin phoneme inventories. Arabic pharyngeal, emphatic, and uvular consonants would require a custom articulatory feature table.
- **No tajweed rule modelling:** the framework detects phoneme-level insertion/deletion/substitution only. Tajweed rules involving temporal duration (Madd), assimilation (Idgham), nasalisation (Ghunnah), and point-of-articulation (Makharij) are not addressed.
- **LLM feedback hallucination risk:** GPT-4 generated feedback contains articulatory guidance that may not match the actual detected error, especially when phoneme prediction is incorrect. No hallucination mitigation strategy is proposed.
- **Prompt injection failure with reference phonemes:** providing correct phoneme sequences in the prompt causes the model to copy them instead of detecting deviations (F1 collapsed from 0.55 to 0.33). No solution is offered beyond flagging this as future work.
- **Multi-L1 degradation:** adding speakers from more L1 backgrounds consistently hurts performance. For a global Quran recitation system (learners from 50+ L1 backgrounds), this is a significant challenge.
- **Comprehensibility vs. helpfulness trade-off:** articulatory terminology improves helpfulness but reduces comprehensibility. For non-specialist Quran learners, this trade-off must be managed with simplified language wrappers.
- **GPT-4 accessibility:** the best-performing feedback system depends on GPT-4, which has regional availability constraints and API cost limitations for deployment.

7.6 Relevance to Your Research

Key Takeaways

- **Use the cascaded framework for tajweed:** Framework 2 (acoustic MDD + LLM feedback) is more directly applicable than the end-to-end LLM. Your tajweed detection model handles rule identification; an LLM layer then generates explanatory feedback. This cleanly separates the technically hard problem (tajweed detection) from the feedback generation problem (LLM strength).
- **Articulatory features → Tajweed attribute features:** the 8-dimensional articulatory feature vector is the most important transferable idea. For tajweed, replace (or extend) this with a **tajweed attribute vector** encoding: articulation point (Makharij), nasalisation level (Ghunnah), duration class (Madd type), voicing (Jahr/Hams), and assimilation target (Idgham partner). Then an LLM can generate feedback like "Your Noon Sakinah was not assimilated — close the mouth slightly and sustain the nasal resonance for 2 counts."
- **Viterbi > Levenshtein for tajweed error localization:** tajweed rules are phonetically non-uniform (e.g. Ikhfa' is acoustically closer to Idgham than to Izhaar). Replace Levenshtein with Viterbi using a tajweed phoneme confusion matrix as the transition probability table, enabling phonetically weighted error alignment.
- **Adopt the Splitting Process Datasets (SPD) for LLM fine-tuning:** decompose the feedback task into (1) error localisation dataset and (2) rule-specific feedback dataset. Direct end-to-end fine-tuning fails. SPD enforces chain-of-thought reasoning and significantly outperforms direct fine-tuning.
- **Use A-Score (with tajweed vocabulary) to evaluate feedback:** replace `detail_richness` phonetic terms with tajweed-specific vocabulary ("Madd", "Ghunnah", "Makharij", "Idgham", "Ikhfa"', "Qalqalah", etc.). This creates a tajweed-specific A-Score that better measures whether the LLM is producing rule-aware corrective guidance.
- **Do not provide canonical recitation phonemes directly in the LLM prompt:** this paper's critical negative result shows that giving the model the correct answer causes it to copy rather than detect. For tajweed, provide the rule label and error type, not the target phoneme sequence.
- **Design for single-L1-group cohorts first:** multi-L1 mixing degrades performance measurably. If your system targets a specific community (e.g. African learners, Indonesian learners), train and evaluate on that cohort first before attempting cross-L1 generalisation.
- **Manage comprehensibility vs. helpfulness explicitly:** wrap articulatory/tajweed terminology in learner-friendly language (e.g. "close the back of your throat" rather than "pharyngeal constriction"), or offer two feedback modes: simplified and technical.

Chapter 8

Ahmed et al. (2023) — Arabic Mispronunciation Detection via Two-Level LSTM

Paper Metadata

Title	Arabic Mispronunciation Recognition System Using LSTM Network
Authors	Abdelfatah Ahmed, Mohamed Bader, Ismail Shahin, Ali Bou Nassif, Naoufel Werghi, Mohammad Basel
Institutions	Khalifa University (UAE); University of Sharjah (UAE)
Journal	*Information* (MDPI), Vol. 14, No. 7, Article 413, 2023
Type	Original model — two-level binary classification (mispronunciation + gender)
DOI	`10.3390/info14070413`
License	Open access, CC BY 4.0

Open Science Status

Code	**Not available** — No GitHub repository found. Paper was written in PyCharm; no supplementary code released.
Dataset	**Not publicly available** — Custom in-house dataset (custom recordings from 53 speakers, 17 countries). Paper states it "can be accessed for research purposes upon request" subject to ethical restrictions. No Kaggle, HuggingFace, or public repository found.

Important Scope Note

This paper targets **general Arabic phoneme mispronunciation** (13 letters), not tajweed rules specifically. However, its MFCC + LSTM architecture and two-level classification framework are directly applicable to Quranic tajweed MDD. The letters targeted overlap with the articulation challenges in tajweed (e.g. , , , , ,).

8.1 Objective

Develop a **speaker-independent, text-dependent** system that:

1. Detects whether an Arabic letter is pronounced correctly or incorrectly (binary mispronunciation classification).
2. Simultaneously identifies the gender of the speaker (binary gender classification).

This is the **first work** to propose two-level classification combining mispronunciation detection with speaker gender recognition for Arabic. The motivation is that gender may influence pronunciation patterns and enable more personalised CALL feedback.

8.2 Dataset

A custom dataset collected specifically for this study — no standard benchmark was available that matched the requirements.

Table 8.1: Dataset composition

Property	Value
Training speakers	30 native Arabic speakers (15M / 15F)
Testing speakers	53 total (11 native + 42 non-native; 23M / 29F)
Countries	17 different nationalities
Age range	9 to 65 years
Training utterances	7,800 (30 speakers × 13 letters × 5 reps + 30 speakers × 39 words × 5 reps)
Testing utterances	3,120
Sampling rate	44.1 kHz, 16-bit
Recording setup	Microphone isolation shield; PRAAT for silence removal
Labels	Binary per utterance: correct / incorrect + gender label: male / female
Availability	Private; available on request (ethical restrictions)

Preprocessing: PRAAT software used to extract speech-containing segments from surrounding silence. The signal model is:

$$x(n) = s(n) + d(n)$$

Table 8.2: Arabic letters targeted (most commonly mispronounced)

Letter	IPA	Positions tested
	/Z/	alone, word-initial, word-medial, word-final
	/h./	alone, word-initial, word-medial, word-final
	/x/	alone, word-initial, word-medial, word-final
	/ð/	alone, word-initial, word-medial, word-final
	/r/	alone, word-initial, word-medial, word-final
	/s/	alone, word-initial, word-medial, word-final
	/S./	alone, word-initial, word-medial, word-final
	/d/	alone, word-initial, word-medial, word-final
	/t/	alone, word-initial, word-medial, word-final
	/D/	alone, word-initial, word-medial, word-final
	/G/	alone, word-initial, word-medial, word-final
	/q/	alone, word-initial, word-medial, word-final
	/k/	alone, word-initial, word-medial, word-final

where $s(n)$ is the clean speech and $d(n)$ is noise. All silent regions are stripped before feature extraction.

8.3 Approach & Architecture

8.3.1 Two-Level Classification Framework

```
Input Signal
  --> Speech Preprocessing (silence removal, denoising via PRAAT
     )
  --> Feature Extraction (MFCC + ZCR + Delta-MFCC + etc.)
  --> LSTM Layer 1 (179 units) + Dropout
  --> LSTM Layer 2 (179 units) + Dropout
  --> Time-Distributed Dense Layer
  --> Flatten
  --> Dense (64) + Dense (1)
        |                |
   Correct/Incorrect   Male/Female
   (mispronunciation)  (gender)
```

Listing 8.1: Overall system pipeline

8.3.2 Feature Extraction

Seven feature types were extracted and compared experimentally. MFCC was found to be the highest-performing feature across all letters:

Feature ablation result: MFCC consistently achieved the highest accuracy across all

Table 8.3: Speech features extracted and compared

Feature	Description
MFCC	Standard 13-coefficient cepstral representation
ΔMFCC	First-order time derivative of MFCCs (delta coefficients)
Δ^2MFCC	Second-order derivative (acceleration coefficients)
ZCR	Zero-crossing rate (dominant frequency indicator)
LPCC	Linear Prediction Cepstral Coefficients
GFCC	Gammatone-Frequency Cepstral Coefficients
LFCC	Log-Frequency Cepstral Coefficients

tested letters. Delta-MFCC and Δ^2MFCC added useful dynamic information. GFCC and LFCC showed lower performance on this task.

8.3.3 LSTM Architecture & Hyperparameter Optimisation

Hyperparameters were tuned using a combination of **grid search** and **manual MSE-minimisation experiments**. Both methods converged to the same optimal values:

Table 8.4: Final optimised network hyperparameters

Parameter	Optimal Value
LSTM layers	2
LSTM units per layer	179
Fully connected layers	2 (64 units, then 1 unit)
Dropout	0.1
Learning rate	10^{-3}
Batch size	16
Epochs	190

The optimisation procedure for each hyperparameter was performed sequentially (neurons → dropout → batch size → epochs), holding all others fixed and selecting the value that minimised MSE on the validation set.

The full architecture:

```
Input: MFCC sequence (T frames x features)
  --> LSTM (179 units) + Dropout (0.1)
  --> LSTM (179 units) + Dropout (0.1)
  --> Time-Distributed Dense
  --> Flatten
  --> Dense (64)
  --> Dense (1)    -- binary: correct=1 / incorrect=0
                   -- simultaneously: male=1 / female=0
```

Listing 8.2: LSTM network architecture

Training: 53 separate models trained — one per letter per word-position combination (letter alone + 3 positions × 13 letters). Total training time: approximately 18 hours on Intel Core i7-4750HQ, 16 GB RAM, NVIDIA GTX 950M.

8.3.4 Data Split

Table 8.5: Dataset partitioning

Split	Ratio
Training	70%
Testing	20%
Validation	10%

8.4 Limitations

- **Not Quran-specific**: targets general Arabic phoneme articulation disorders, not tajweed rules. The 13 letters are not a direct mapping to tajweed rule categories.
- **Private, non-replicable dataset**: custom recordings with no public release. The first benchmark for this task, but not comparable to existing benchmarks.
- **No code released**: 53 individually trained models, no repository.
- **Training cost**: 18 hours of training for 53 models is not scalable to a full tajweed rule set.
- **Gender as a variable**: the paper found that a separate gender model did not improve mispronunciation accuracy (81.52% with gender vs. 83.77% without), suggesting Arabic mispronunciation patterns may not be distinctly gender-specific.
- **Per-letter variation**: letter (/r/) showed consistently the lowest accuracy across all positions (66–87%), while (/Z/) achieved the highest (82–92%). This position- dependent variation is not modelled or explained architecturally.
- **Lower overall accuracy**: 81.52% average is lower than the tajweed-specific LSTM paper (Al Harere & Al Jallad, Paper 3) which achieved 95–96% — partly because Arabic phoneme MDD is harder than binary per-rule tajweed classification.

8.5 Relevance to Your Research

Key Takeaways

- **Two-level classification is feasible**: a single LSTM network can simultaneously output multiple labels (mispronunciation state + speaker attribute). This is directly applicable to a tajweed system that outputs both rule identity and error type.
- **MFCC remains the best single feature**: among 7 feature types (MFCC, ΔMFCC, Δ^2MFCC, ZCR, LPCC, GFCC, LFCC), MFCC consistently won — confirming the choice in other reviewed papers.
- **Delta and Delta-Delta MFCCs add useful dynamics**: incorporating ΔMFCC and Δ^2MFCC captures temporal dynamics not present in static MFCC coefficients. Consider including these in your feature pipeline.
- **Grid search + manual MSE optimisation**: the dual-method hyperparameter tuning approach is a practical protocol for LSTM-based models; both methods converged to the same values, validating grid search efficiency.
- **Gender had negligible impact**: the two-level model was not improved by gender recognition, suggesting you should not add gender as an auxiliary task unless you have specific domain evidence it helps for tajweed.
- **Word-position matters**: letter-in-word-final position consistently showed lower accuracy than other positions. For tajweed, the context (position of the rule occurrence within the word or verse) should be an explicit feature or architectural consideration.

Chapter 9

Agarwal & Chakraborty (2019) — A Review of CAPT Systems for English

Paper Metadata

Title	A Review of Tools and Techniques for Computer Aided Pronunciation Training (CAPT) in English
Authors	Chesta Agarwal, Pinaki Chakraborty
Institution	Netaji Subhas University of Technology, New Delhi, India
Journal	*Education and Information Technologies* (Springer), Vol. 24, No. 6, November 2019, pp. 3731–3743
Type	Literature review — no original model proposed
DOI	10.1007/s10639-019-09955-7

Scope Note — Why This Paper Is Included

This paper is **not about Quranic recitation or Arabic speech**. It surveys CAPT (Computer Aided Pronunciation Training) systems for English. Its value to your project is **architectural and pedagogical**: it maps the design space of pronunciation training systems, identifies what makes feedback effective, and documents the progression from rule-based to deep-learning approaches. The feedback design principles and system pipeline patterns are directly transferable to a tajweed CAPT system.

Open Science Status

Code	**Not applicable** — pure literature review, no model implemented.
Dataset	**Not applicable** — no data collected or released.
Note	Paper is behind Springer paywall; not open access. Widely cited (100+ citations); available via ResearchGate.

9.1 Objective

Classify and survey the full landscape of CAPT systems developed for English pronunciation training, identify state-of-the-art practices, and recommend directions for future development. The paper organises all reviewed systems into four technology-based categories and evaluates them on both technological performance metrics and human learning outcomes.

9.2 Dataset

Not applicable — this is a review paper. No dataset is collected or used. The reviewed systems span publications from 1972 to 2017 and were sourced from IEEE, ACM, and linguistics conference proceedings.

9.3 Approach & Architecture — CAPT System Taxonomy

The paper proposes a four-category taxonomy of CAPT systems based on the primary technology used. These categories form a maturity spectrum from naive to professional learners:

Table 9.1: Four categories of CAPT systems (Agarwal & Chakraborty 2019)

Category	**Mechanism**	**Strengths**	**Target users**
Visual Simulation	Records speech; provides feedback via images, animated characters, videos	Simple, attractive, accessible to hearing-impaired	Young / naive learners
Game Based	Voice-command game; correct pronunciation required to progress	Personalised; simulates real conversation; high engagement	School-age / informal
Comparative Phonetics	Stochastic analysis comparing native language phonemes with target language	Leverages existing phonological knowledge; handles L1 interference	Adult, fluent L1
ANN Based	Deep neural network trained on expert recordings; detects and diagnoses mispronunciation	Highest accuracy; handles complex error patterns	Experienced / professional

9.3.1 Typical CAPT Pipeline (all categories)

All reviewed CAPT systems follow the same four-stage pipeline regardless of the technology used:

```
Stage 1: RECORD
  --> Capture learner's speech utterance

Stage 2: DETECT
  --> Identify whether a mispronunciation is present
  --> Binary: correct / incorrect

Stage 3: DIAGNOSE
  --> Classify the type of error
  --> Substitution / Deletion / Insertion / Phoneme-specific
      error

Stage 4: FEEDBACK
  --> Present corrective guidance to the learner
  --> Visual / audio / animated / score-based
```

Listing 9.1: Universal CAPT system pipeline

The key gap identified across all categories: **most systems stop at Stage 2 (detection)** and do not implement Stage 3 (diagnosis) or Stage 4 (actionable feedback). This is the same gap documented in the Quranic recitation literature.

9.3.2 ANN-Based Systems — Technical Detail

The ANN-based category is most relevant technically. Key systems reviewed:

Table 9.2: ANN-based CAPT systems reviewed

Work	Architecture	Key contribution
Akima et al. (1992)	Basic ANN	First neural net for pronunciation detection
Qian et al. (2012)	DBN-HMM + DNN	Stochastic + deep learning for MDD
Chen & Jang (2015)	ANN + SVM	Phone-based word recognition; auto-grading
Wang & Lee (2015)	DNN (stochastic)	Emphasis on frequently repeated mistakes
Li et al. (2017)	Multi-dist. DNN	Models 3 causes of mispronunciation

9.3.3 State-of-the-Art Practices Identified

The paper identifies four practices consistently used in the highest-performing systems:

1. **Pronunciation dictionaries** — a database of acceptable and frequently occurring incorrect pronunciations, used for comparison-based mispronunciation detection.

2. **Deep learning** — multiple hidden layers in ANN; consistently outperforms shallow probabilistic models.
3. **Multimedia feedback** — images, animation, video clips to show learners how to correct pronunciation; critical for actionable guidance.
4. **Automated grading** — correlation with human expert grades shown to be high (Chen & Jang 2015); reduces teacher workload.

9.3.4 Measured Performance Across Systems

Table 9.3: Selected performance benchmarks from reviewed CAPT systems

Work	Tech. metric	Human outcome
Wang et al. (2008)	86% mispronunciation detection accuracy	—
Li et al. (2017)	5% false positives; 31% false negatives	Phoneme errors: 17% → 11%
Jing & Yong (2014)	—	23% improvement in pronunciat
Akima et al. (1992)	—	7% mispronunciation decrease a
Chen & Jang (2015)	15% false mispronunciation rate	Strong correlation with expert g

Overall field benchmark: best systems detect up to 86% of mispronunciations and can reduce learner errors by up to 23%.

9.4 Limitations

- **English-only scope**: all reviewed systems target English; the Arabic/Quranic domain has fundamentally different phonological challenges (pharyngeal consonants, emphasis, elongation rules) not addressed here.
- **2019 cutoff**: the review predates the Transformer/SSL era (wav2vec 2.0, HuBERT, Whisper). All ANN-based systems reviewed use pre-Transformer architectures.
- **Pedagogical depth still lacking**: even the best-performing systems stop at detection or simple grading. None provide the kind of rule-specific, corrective, multi-step feedback that the paper recommends as future work.
- **No Arabic CAPT coverage**: the paper acknowledges CAPT exists for Arabic (citing Abdou et al. 2006) but does not survey it — a gap that positions your Quranic tajweed work as filling unexplored territory.
- **No code or reproducibility**: review paper; no implementations produced or released.

9.5 Relevance to Your Research

Key Takeaways

- **The four-stage pipeline is your system blueprint**: Record → Detect → Diagnose → Feedback. All reviewed tajweed papers (Papers 1–6) implement only Stages 1–2. Your system's differentiator is fully implementing Stages 3 and 4.
- **ANN-based = highest accuracy**: across all technology categories, deep learning consistently dominates. This validates your choice of a neural-network based approach over rule-based or probabilistic methods.
- **Multimedia feedback matters for learning outcomes**: detection alone does not improve pronunciation. Actionable visual/audio feedback is what produces measurable improvement (23% in Jing & Yong 2014). Your system should output more than a binary label.
- **Pronunciation dictionaries are a practical tool**: a structured database of correct and incorrect tajweed patterns (one per rule) could serve as a comparison reference, analogous to the pronunciation dictionaries used in English CAPT.
- **Automated grading has high agreement with human experts**: building a scoring module that correlates with sheikh/teacher assessment would be a strong validation metric for your system.
- **Use this paper to justify your introduction**: Agarwal & Chakraborty (2019) provides a well-cited, peer-reviewed framing for why CAPT systems are valuable and why deep learning is the correct technological choice — directly applicable to your introduction and related works section.